

Generative Adversarial Network (GAN)

Training Report MNIST Handwritten Digit Generation

Framework: TensorFlow 2.x with Keras API

Dataset: MNIST – 60,000 grayscale handwritten digit images (28×28)

Epochs: 20 | Batch Size: 128 | Noise Dim: 100

Optimizer: Adam | LR = 0.0002 | $\beta_1 = 0.5$

1. Objective and Brief Explanation of GANs

The primary objective of this assignment is to implement a Generative Adversarial Network (GAN) from scratch using TensorFlow 2.x and train it on the MNIST dataset to learn the underlying distribution of handwritten digits, enabling the model to synthesise realistic new digit images from random noise vectors.

Generative Adversarial Networks, introduced by Ian Goodfellow et al. in 2014, consist of two neural networks — a Generator (G) and a Discriminator (D) — trained simultaneously in an adversarial min-max game:

- **Generator:** Maps a latent noise vector $z \sim N(0, I)$ to a synthetic data sample $G(z)$, aiming to fool the discriminator into classifying fake images as real.
- **Discriminator:** Receives either a real image x or a fake image $G(z)$ and outputs $D(x) \in [0, 1]$ — the probability of the input being real.
- **Training Objective:** $\min_G \max_D V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$

At convergence, the generator produces samples indistinguishable from real data, and the discriminator outputs 0.5 for every input. This project uses Binary Cross-Entropy (BCE) as the loss function and Adam optimizer with parameters tuned for stable GAN training (lr = 0.0002, $\beta_1 = 0.5$).

2. Architecture Summary

2.1 Generator Architecture

The generator uses transposed convolutions (fractionally-strided convolutions) to upsample a 100-dimensional noise vector into a 28×28 grayscale image.

Layer	Configuration	Output Shape
Dense	7×7×256 units, no bias	(None, 12544)
BatchNorm + LeakyReLU	—	(None, 12544)
Reshape	—	(None, 7, 7, 256)
Conv2DTranspose	128 filters, 5×5, stride 1, same, no bias	(None, 7, 7, 128)
BatchNorm + LeakyReLU	—	(None, 7, 7, 128)
Conv2DTranspose	64 filters, 5×5, stride 2, same, no bias	(None, 14, 14, 64)
BatchNorm + LeakyReLU	—	(None, 14, 14, 64)
Conv2DTranspose (Output)	1 filter, 5×5, stride 2, same, Tanh	(None, 28, 28, 1)

2.2 Discriminator Architecture

The discriminator is a convolutional binary classifier. Dropout layers (rate = 0.3) regularise the network, preventing it from overpowering the generator too early.

Layer	Configuration	Output Shape
Conv2D	64 filters, 5×5, stride 2, same	(None, 14, 14, 64)
LeakyReLU +Dropout(0.3)	—	(None, 14, 14, 64)
Conv2D	128 filters, 5×5, stride 2, same	(None, 7, 7, 128)
LeakyReLU + Dropout(0.3)	—	(None, 7, 7, 128)
Flatten	—	(None, 6272)
Dense (Output)	1 unit, Sigmoid activation	(None, 1)

3. Loss Graphs and Sample Generated Outputs

3.1 Training Loss Curves

(Insert your loss graph here — the one plotted in Step 13 of the code)

The Generator and Discriminator losses were recorded at each epoch and plotted after training. Key patterns observed:

- **Generator loss** starts high (the generator initially produces random noise that the discriminator easily rejects) and gradually decreases as it learns to fool the discriminator.

- **Discriminator loss** starts moderate and stabilises as both networks reach a dynamic equilibrium.

3.2 Sample Generated Images

(Insert your generated image grid here — from Step 14 of the code)

The images above were produced by the trained generator from random noise vectors at epoch 20. Progression across epochs:

- **Epoch 1–5:** Uniform grey blobs with no recognisable structure.
- **Epoch 6–10:** Blob boundaries sharpen; rough digit contours begin to appear.
- **Epoch 11–15:** Distinct stroke-like features visible; variety across samples increases.
- **Epoch 16–20:** Clear digit-like images with high contrast and varied handwriting styles.

4. Observations and Challenges

4.1 Key Observations

- **Adversarial equilibrium:** Both networks improved together throughout training. Whenever one network dominated, the other adapted — a hallmark of a well-balanced GAN setup.
- **Loss oscillation:** Generator and discriminator losses oscillated rather than smoothly decreasing, which is entirely normal in GAN training.
- **Progressive quality improvement:** Generated images visibly improved with each epoch. By epoch 15, outputs clearly resembled MNIST digit shapes.
- **Tanh normalisation:** Normalising images to $[-1, 1]$ for the generator's Tanh output activation, and de-normalising for display ($\times 127.5 + 127.5$), was essential for correct image visualisation.

4.2 Challenges Faced

- **Training instability:** GANs are notoriously difficult to train. The hyperparameter choices from the DCGAN paper ($\text{lr} = 0.0002$, $\beta_1 = 0.5$) were critical — higher learning rates caused the discriminator to overpower the generator entirely.
- **Mode collapse risk:** Without Dropout in the discriminator, the generator tended to produce only a narrow set of very similar outputs. Dropout(0.3) was essential to maintain output variety.

- **Batch size sensitivity:** A batch size of 128 provided a balance between gradient variance and training speed. Smaller batches led to noisy gradients and training instability.
- **Epoch count trade-off:** 20 epochs was sufficient for recognisable digits on MNIST, but more epochs (50–100) with learning rate scheduling would yield sharper, higher-quality outputs.

5. Conclusion

This project successfully implemented a Deep Convolutional GAN (DCGAN) in TensorFlow 2.x to generate synthetic MNIST handwritten digit images. The generator learned to map 100-dimensional noise vectors to realistic 28×28 digit images by training adversarially against a convolutional discriminator. Key design decisions — BatchNormalisation, LeakyReLU, Dropout, Adam with $\beta_1 = 0.5$, and Tanh output activation — collectively ensured stable and effective training across 20 epochs. The loss curves and generated samples confirm that the model successfully captured the data distribution of MNIST.

Future improvements could include conditional GAN (cGAN) variants for class-controlled generation, progressive growing, spectral normalisation, or applying the architecture to higher-resolution and more complex datasets.